<u>Team Todog:</u>

- **(105910780) Giuliano Zuccara –** was aiming to complete the Player logic, this includes the main scene and the code, both Enemies these being the slimes and goblins with their codes, scenes and respawn logic, and made and some extra weapons that the player can obtain through leveling up e.g. the Summoned Sword and mimic. Also wanted to do some art stuff which was achieved through editing player sprites and making the summoned sword in Krita. Finally worked on making a base for the combat system within the game these being auto attacks that the player does and enemy damage logic.

- **(104545479) Lucas Freestone –** Was aiming to tackle most of the visuals and art for the game. Everything from concept art to final sprite and tile work for the enemies and environment, including the game's logo on the title screen. The player, slime and environment sprites/tile-sets originally came from a template which were heavily edited to resemble what is in game now. The goblins use the player sprites as a base.

- **(105917529) Jonathan Whitford -** Was aiming to work on a broad range of tasks for the game including; communication setup and management, task/time tracking (ei discord server and 'clickup' project, play tested prototypes of the game, tile art (environment) in adobe photoshop, map layout/planning in google slides, level transition, creating both levels (this was changed as went with an open infinite world), added main menu background & music as well as background music to main level. I had intended to incorporate the tile sets into the game environment, but I was unable to do so because the game is evolving into a larger open world experience and the tile sets are difficult to incorporate into an apparently limitless area.

- **(105348873) Hunter Burfitt –** Was aiming to work on the majority of the coding, covering the player logic, experience system, enemy logic, the various weapons, and the objectives. Additionally did some simplistic art for some assets such as the sprites and particles for a few weapons (fireball, light orb, icicles), the enemy damage particles, the objective indicators, and the looping background texture. These art assets were all made using the built-in godot sprite creation tools, mostly the 2D Gradient creator. I was planning on adding a system where enemies would drop experience orbs for the player to collect instead of the player directly obtaining experience, but it was not feasible with the time limit.

Controls for GoblinFall: Survival:

Due to this game being similar to Vampire survivors the controls are very simple these being mainly movement and a mouse click on occasion. See below for details:

Keyboard Controls:

Movement – W, A, S, D with arrow keys as another option.

Dodge – Shift or space – allows the player to be invulnerable at the cost of not attacking

Level up menu – mouse click on desired upgrade e.g fireball, ball of light or summoned sword.

Tools used During Game production:

- Discord for communication
- ClickUp for time and task management
- Godot for our game engine (and for some visuals these visuals being for the fireball and ball of light scenes)
- GitHub for version control, which we utilised to commit and push new changes that we added to the game so all members can have an up to date version.
- Krita (Visuals)
- Photoshop (for tile sprites)
- Pixelorama (pixel art and animation tool for the enemy sprites)

Game Guide:

The game itself as mentioned above has very little controls mainly movement with W,A,S,D or arrow keys and with the occasionally mouse click to select an upgrade.

The game itself for gameplay is simple its similar to Vampire Survivors but has some Dungeon Crawler elements to it the main aim of the game is to defeat the Goblin Brute and not fail while doing it, during this gaol the player will level up and obtain abilities, some of these being a mimic or an orbiting sword to name a few.

Resources used:

https://www.youtube.com/watch?v=MKsr119jyzs
https://www.youtube.com/watch?v=7sxEF-YtAqU

These links were used to help with the enemy spawner logic; the top video showcases the base code where only one enemy can be spawned while the bottom video showcases it being used with an array to summon multiple types of enemies.

https://www.youtube.com/watch?v=v3v01p26NP0&list=PL3cGrGHvkwn0zoGLoGorwvGj6dHCjLaGd&index=8

This link was used to help us create changing scenes within our game world in order to allow for multiple different areas within the game e.g a field and a dungeon.